

Word Embedding & Sequential Learning

Dongwook Choi

Lang&AGI Lab, Yonsei University

2025. 06. 16

Contents

1. Welcome to the NLP!

2. Word Embedding

3. Sequential Learning

- RNN & LSTM
- Seq2Seq
- Attention

실습 코드

<https://drive.google.com/drive/folders/16rotE2B2uy2te5xxVBcN9SuvqwfTnNOv?usp=sharing>

Welcome to the NLP!

Welcome to the NLP!

- Natural Language (자연어)
사람들이 일상적으로 쓰는 언어
- Natural Language Processing (NLP)
컴퓨터가 인간의 언어를 이해하고 처리할 수 있도록 하는 기술 분야

Welcome to the NLP!

- Natural Language (자연어)
사람들이 일상적으로 쓰는 언어
- Natural Language Processing (NLP)
컴퓨터가 인간의 언어를 이해하고 처리할 수 있도록 하는 기술 분야
- 주요 응용 분야
 1. 기계 번역
 2. 감정 분석
 3. 챗봇
 4. 텍스트 요약

...

Welcome to the NLP!

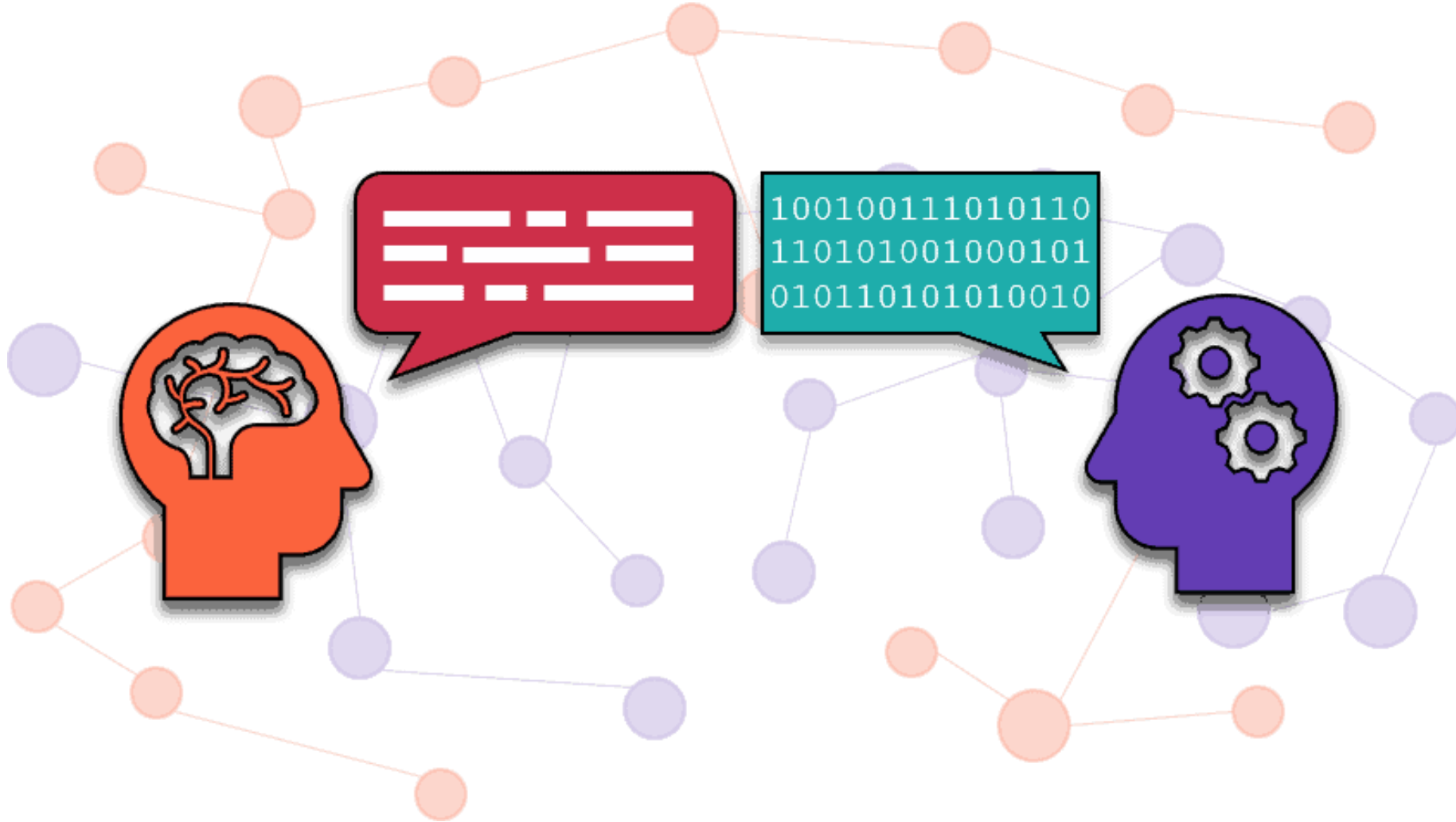
ChatGPT 4o ▾

안녕~

안녕하세요! 반갑습니다 :)
무엇을 도와드릴까요?



Welcome to the NLP!



Word Embedding

Word Vector

- 컴퓨터가 자연어를 처리하려면?

우선, 텍스트로 이루어진 자연어를 컴퓨터가 이해할 수 있는 숫자로 바꿔야 함

→ 우선, 자연어의 기본 단위인 단어를 벡터로 표현해보자

→ **Word Embedding**

Word Embedding

- Word Embedding란?
 - '단어의 의미'를 벡터로 표현하는 것

Signified (idea or thing)



Signifier (symbol)



“사자”

[0.1, 3.5, 1.3, ..., 5.6]

One-hot Encoding

- 먼저 0과 1 만을 사용하여 단어들을 나타내어보자
- 단어마다 고유한 인덱스를 부여하여, unique value로 단어를 표현

[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0]

= Company

One-hot Encoding

- 문제점

- 단어의 양이 늘어나면 **벡터의 차원**이 늘어남 (차원 = 단어의 총 개수)

```
vocab = ['king', 'queen', 'food', 'milk']
```

```
'king'의 One-Hot Encoding: [1. 0. 0. 0.]  
'queen'의 One-Hot Encoding: [0. 1. 0. 0.]  
'food'의 One-Hot Encoding: [0. 0. 1. 0.]  
'milk'의 One-Hot Encoding: [0. 0. 0. 1.]
```

- 대부분의 요소가 0인 벡터로, **메모리 비효율적**임 -> Sparse Representation
- 단어 간의 **의미적 유사성**을 제대로 반영하지 못함

```
'king'과 'queen'의 유사도 (One-Hot): 0.0  
'king'과 'food'의 유사도 (One-Hot): 0.0  
'king'과 'milk'의 유사도 (One-Hot): 0.0
```

Good Word Vector?

- How can we solve it?
 - 단어를 **dense한 vector**로 표현
 - **벡터의 차원**을 무한정 늘리면 안됨
 - 벡터의 **모든 숫자는 의미를 가져야 함**
- 각 벡터는 단어의 의미를 내포해야 함



Word Embedding

- 단어 간의 **의미적 유사성**을 나타내도록 하기
- One-hot encoding과 비교하였을 때의 장점?
 1. 대부분의 값이 0이 아닌 **dense**한 벡터로 표현해, **적은 차원의 벡터**로 단어를 표현할 수 있음
 2. 의미적으로 유사한 단어들을 **유사한 벡터**로 표현할 수 있음

Ex) Word2Vec, GloVe 등

$$\begin{array}{l} \text{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \end{array} \quad \begin{array}{l} \text{monetary} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix} \end{array}$$

Word Embedding

- Dense representation

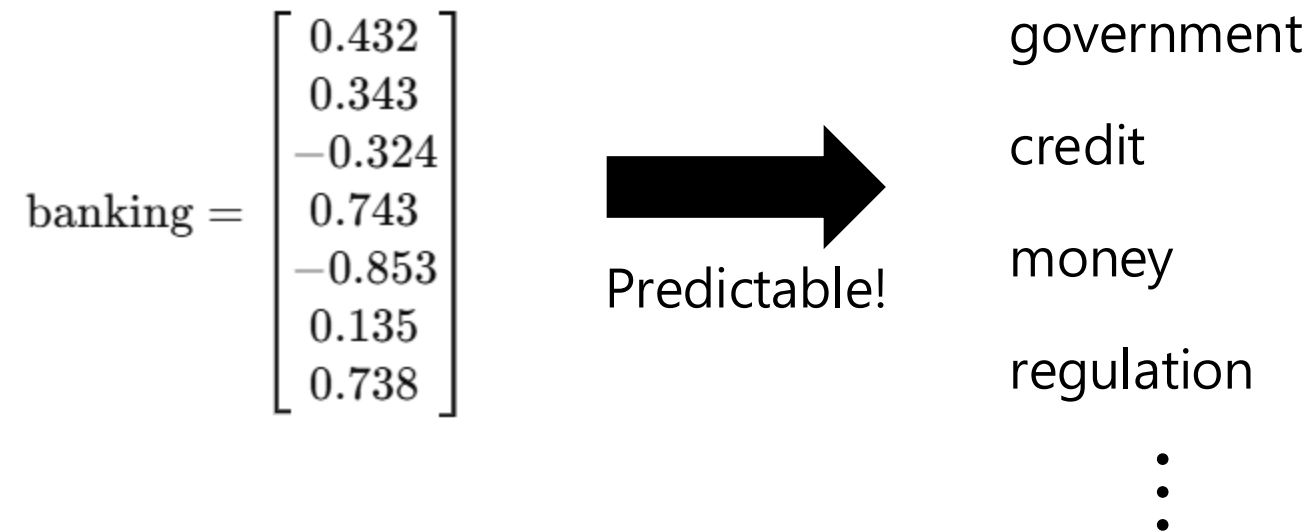
- 단어를 n차원 dense vector로 나타낸다 = 단어의 의미를 n개의 특징으로 표현

$$\text{dog} = \begin{bmatrix} 0.288 \\ -0.192 \\ 0.423 \\ -0.782 \\ \dots \\ 0.243 \end{bmatrix}^{200(\text{dim})}$$

$$\text{cat} = \begin{bmatrix} -0.356 \\ -0.643 \\ 0.234 \\ 0.953 \\ \dots \\ -0.845 \end{bmatrix}^{200(\text{dim})}$$

Word Embedding

- 그러한 벡터를 어떻게 만들까...?
- Can we learn the representation using context?

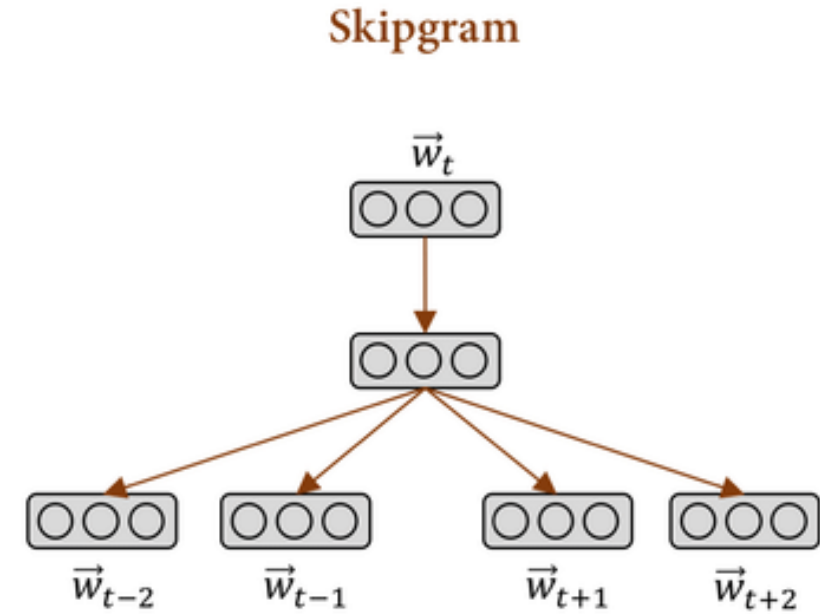
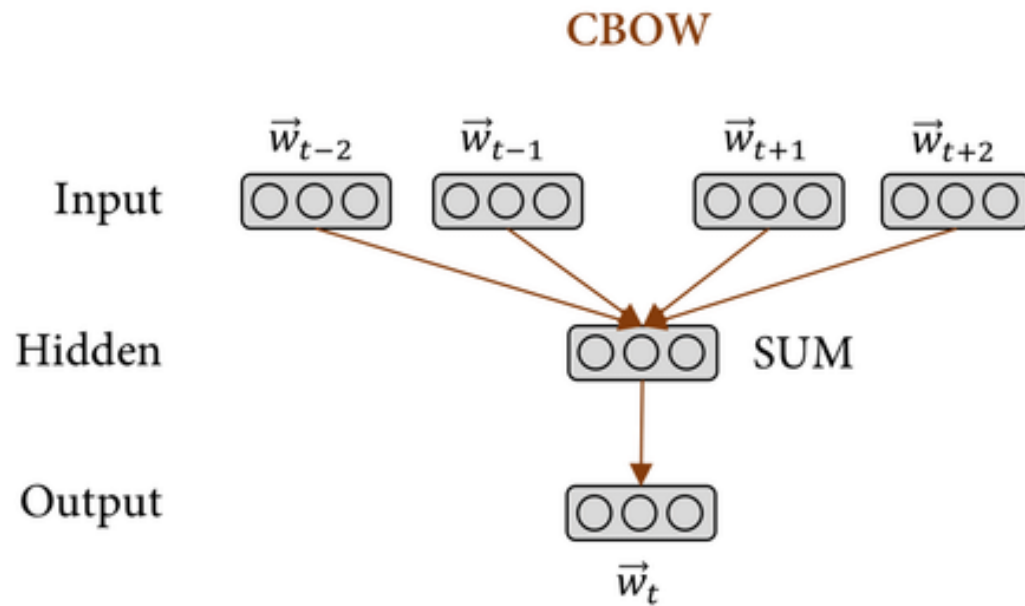


Word Embedding

- Examples
 - Word2Vec - CBOW, Skip-gram
 - GloVe

Word2Vec

- “특정 단어가 어떤 다른 단어들과 함께 자주 나오는가”
- 즉, 단어가 사용되는 **문맥을 학습** - Ex) CBOW, Skip-Gram



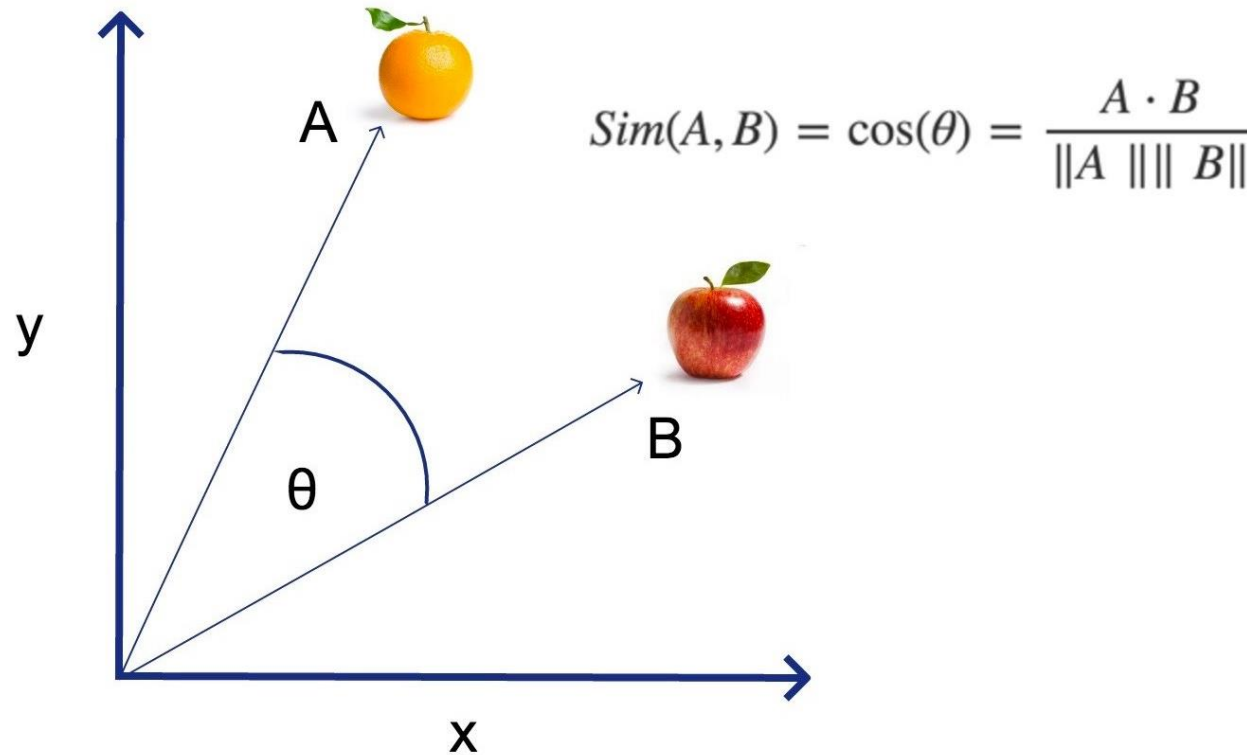
GloVe

- “단어들이 얼마나 자주 함께 나오는가”
- 즉, 단어 쌍의 동시 등장 빈도수(co-occurrence)를 학습

	cat	fast	hat	in	no	ran	the	wears
cat	0							
fast	2	0						
hat	2	2	0					
in	1	1	1	0				
no	1	1	1	0	0			
ran	1	1	1	1	0	0		
the	3	3	3	2	1	2	1	
wears	1	1	1	1	1	0	1	0

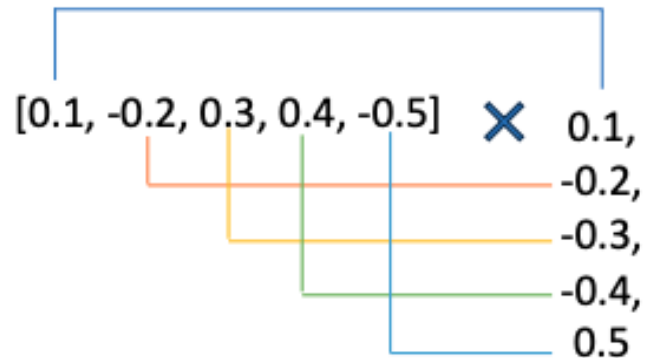
Cosine Similarity

- Word Embedding을 통해 n차원 vector로 표시된 word 간의 의미적 유사성 계산 가능



Word Embedding

- 단어 간의 Similarity?
= 단어들이 얼마나 semantically(의미적으로) 유사한가?



$$\begin{aligned} &= (0.1 \times 0.1) + (-0.2 \times -0.2) + (0.3 \times -0.3) + (0.4 \times -0.4) + (-0.5 \times 0.5) \\ &= 0.01 + 0.04 - 0.09 + 0.16 - 0.25 \\ &= -0.13 \end{aligned}$$

Practice 1

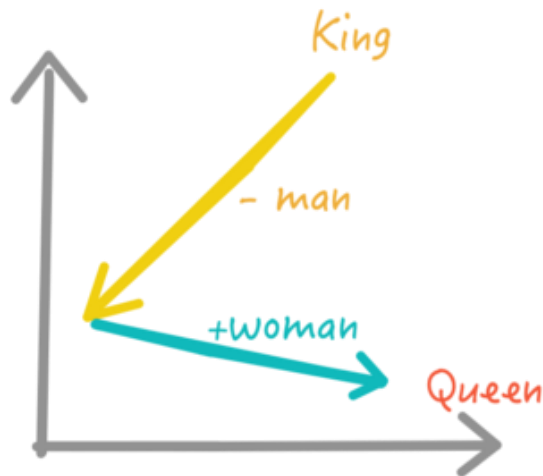
- **One-hot encoding 실행**해보기
- 목표
 - Cosine similarity 구현해보기
 - One-hot encoding 된 단어들 간의 의미적 유사도를 계산
 - One-hot encoding 차원의 문제점 확인

Practice 2

- **Word2Vec, GloVe 실행해보기**
- 목표
 - Word2Vec, GloVe를 통해 단어의 유사도를 계산
 - 'Dimension'이 무엇을 뜻하는지 one-hot encoding과 비교하여 생각해보기

Practice 3

- Word Embedding 추가 실습
- 목표
 - a)의 예시처럼, 단어 간 관계 실험 (analogy test. Ex) king – man + woman = ?)
 - b)의 예시처럼, 유사한 단어 쌍 간의 vector 차이를 기반으로 similarity check



a) Learns Analogy



b) Similar Words have same angles

Task 1

- 목표
 - GloVe 모델로 주어진 단어 목록에서 **의미적으로 관련이 적은 단어**를 찾아내어 보기
 - 여러 개의 단어들 중에서 **의미적으로 유사한 단어들을 올바르게 연결**해보기

Word Embedding

- Examples
 - Word2Vec - CBOW, Skip-gram
 - GloVe
 - ELMO
 - BERT



Word Embedding

- Word2Vec, Glove → *outdated...*
- BERT (Devlin et al. 2018) 
 - Bidirectional Encoder Representations from Transformers
 - 같은 단어라도 문맥에 따라 다른 **vector**로 표현 가능
 - He went to the **bank** to deposit money (금융 기관)
 - She sat by the **bank** of the river (강가)

Task 1

- 목표
 - **BERT 모델을 실행한 후, 앞서 사용한 Word2Vec, GloVe 기반 word embedding과의 차이를 직접 비교하고 분석해보기**

How about sentence?

- 컴퓨터가 단어만 이해하면 되나? → No, 문장을 이해할 수 있어야 함
- How? **Tokenization**

Tokenization

- Text를 smaller units (tokens)으로 쪼개는 과정을 의미
- Tokens?
 - Words, subwords, characters, etc...
 - Raw text를 computer가 이해할 수 있는 단위로 변환한다.

Tokenization

1. Word Tokenization

- Splits text into words
- "I love NLP!" → ["I", "love", "NLP", "!"]

2. Subword Tokenization

- Breaks words into smaller units
- "unhappiness" → ["un", "happiness"]

3. Character Tokenization

- Treats each character as a token
- "hello" → ["h", "e", "l", "l", "o"]

Tokenization

- Now? BPE (BytePair Encoding)-based
- How?
 - 1. Initial state:** Split words into individual characters
e.g., learning → l e a r n i n g
 - 2. Merge the most frequent character pairs**
e.g., i n → in, le ar → lear, ...
 - 3. Repeat until reaching the pre-defined subword vocabulary**
e.g., learning → learn ing

Llama, GPT도 BPE 기반 & 유사한 tokenizer 사용!

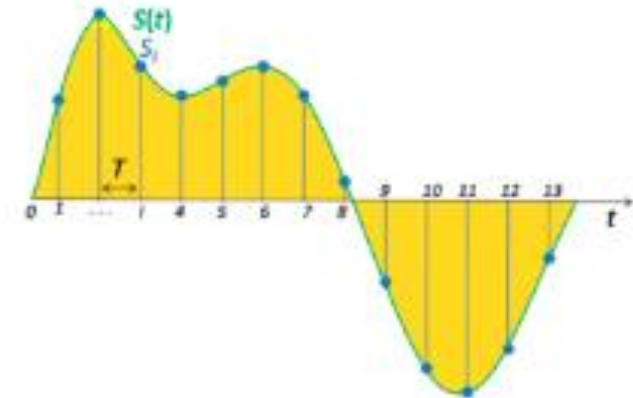
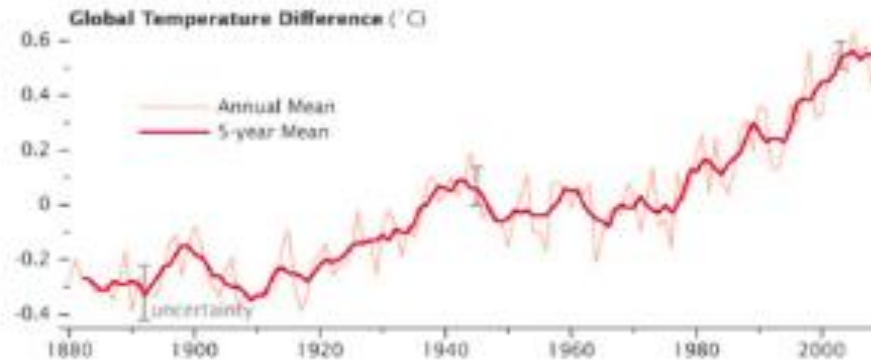
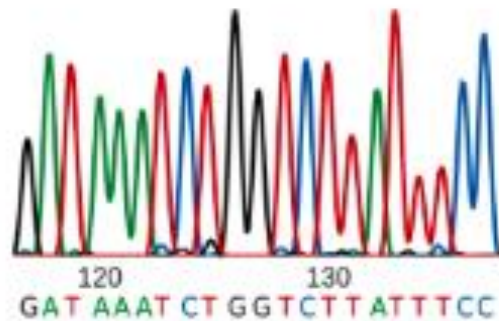
Sequential Learning

Sequential Learning

- Is word embedding enough?
 - 문제점
 - 단어가 쓰인 맥락을 무시하고 항상 같은 벡터로 처리
 - "He swung the **bat** and hit a home run." →  야구 방망이
 - "A **bat** flew out of the dark cave." →  박쥐
- ➔ 지금까지 배운 방법만으로는 이 둘을 구분하지 못 함

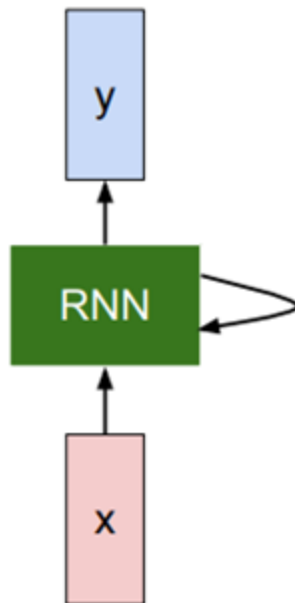
Sequential Learning

- 순차적 데이터 (Sequential Data)
 - 텍스트, 음성 등 순차적으로 발생하는 데이터
 - 이전 데이터가 현재 데이터에 영향을 미침



Sequential Learning

- RNN?
 - Recurrent Neural Networks
 - 순차적인 데이터를 처리하기 위해 설계된 신경망 구조



some function
with parameters W

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state old state input vector at
some time step

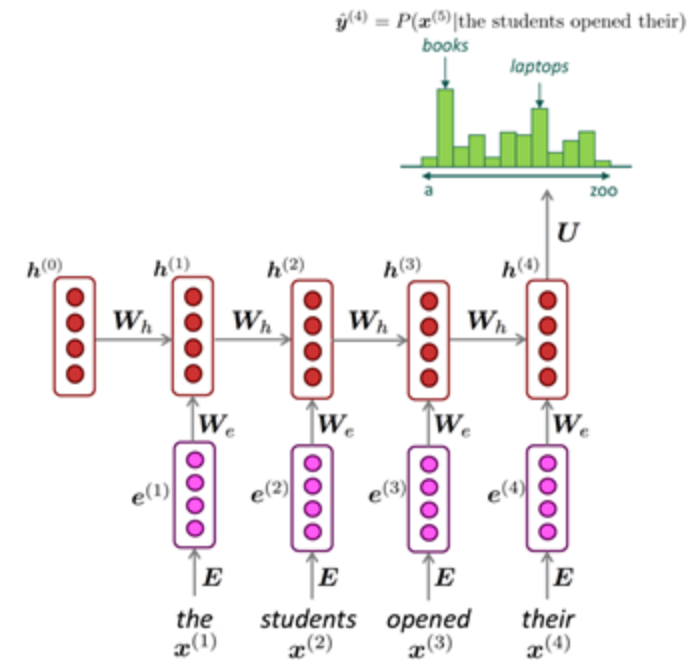
Recurrent Neural Networks

- RNN의 구조

- **Hidden state:** 이전까지의 모든 정보를 저장하는 벡터
- 매 시점마다 새 입력 데이터와 이전 hidden state를 사용해, 새로운 hidden state를 계산

- Recurrent 구조란?

- 하나의 cell이 매 시점마다 반복 호출되는 구조



RNN

- RNN의 특징

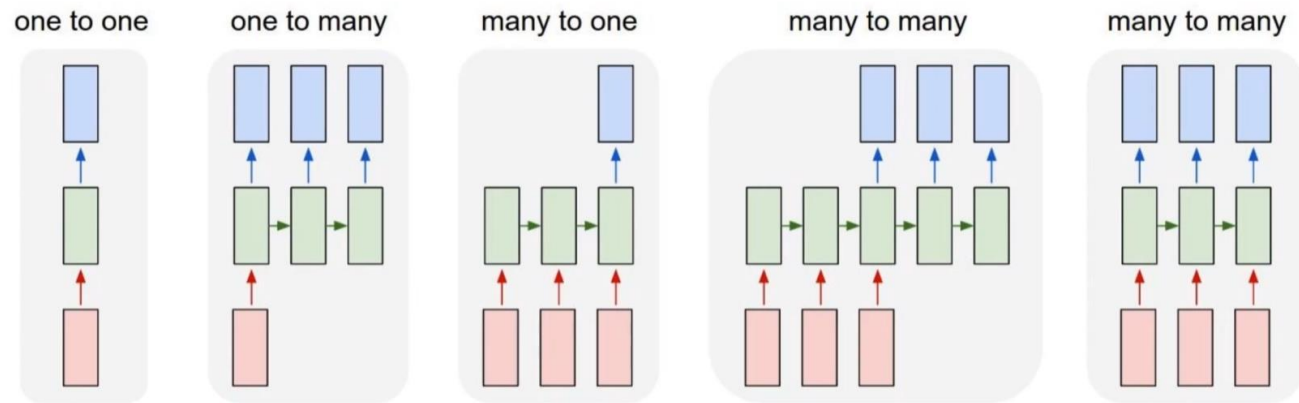
- 입력 **sequence**를 **순서대로 읽어**, 각 시점의 hidden state와 출력을 반복적으로 계산

- 장점

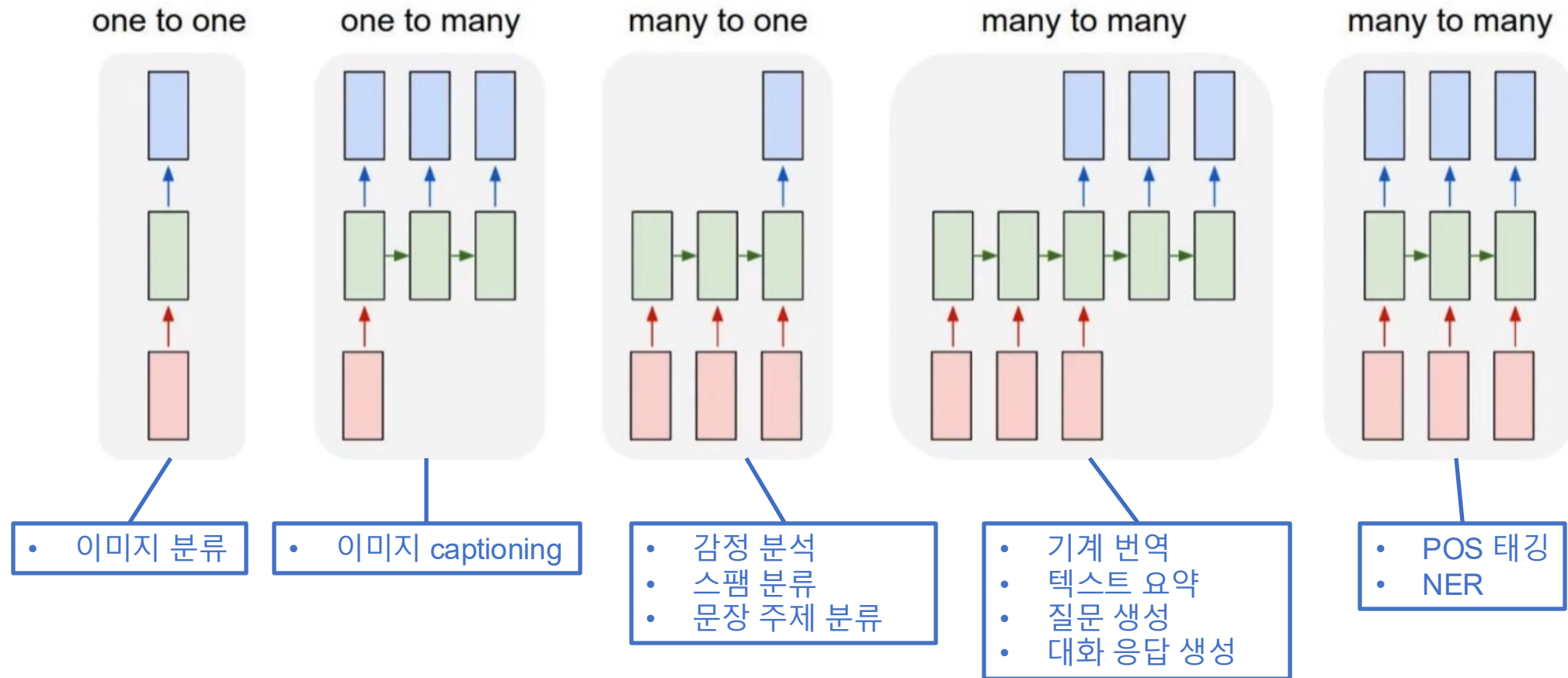
- 어떤 길이의 입력이든 처리 가능
 - 긴 입력 데이터를 처리해도 모델 크기가 증가하지 않음

- 다양한 task에 적용 가능

- Many-to-one
 - One-to-one
 - One-to-many...



RNN

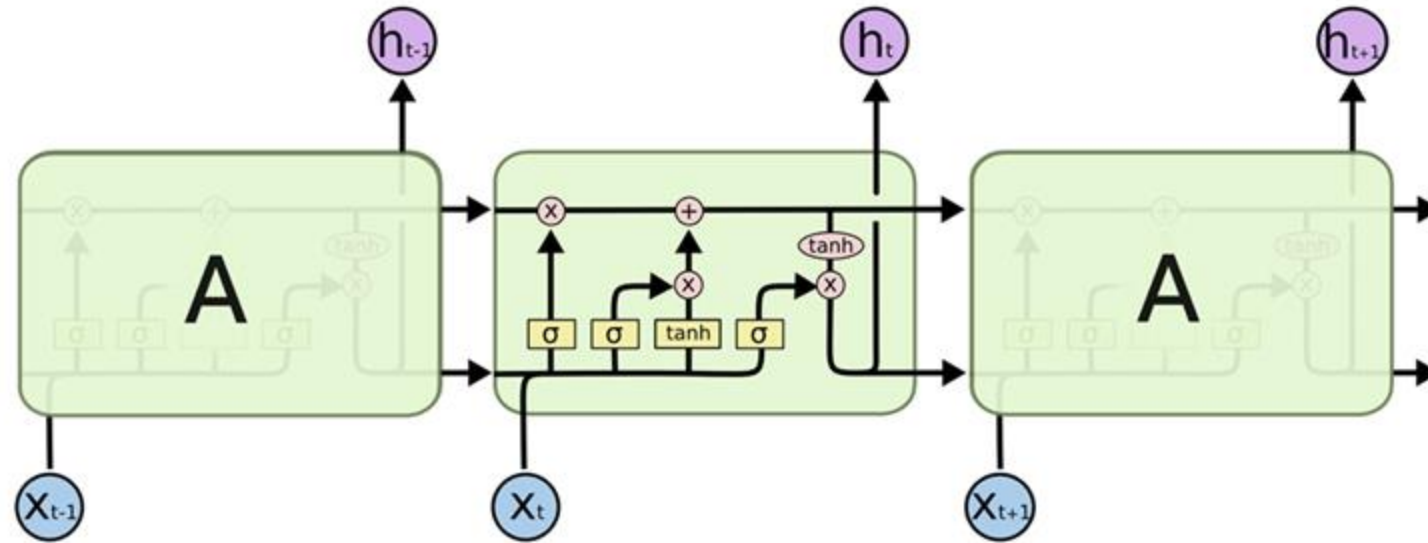


RNN

- RNN의 한계
 - 반복 계산이 느림
 - ★ 장기 의존성 문제(Long-term dependency)
문장이 길어지면 뒤에서 앞의 정보에 접근하기 어려워 짐
→ 중요한 정보가 소실될 수 있음
- RNN 적용 가능 분야
 - NLP: 다음 단어 예측, 기계 번역, 주어진 seed text에서 새로운 text 생성
 - 음성인식: 음성 데이터를 text로 변환
 - 시계열 분석: 주가 예측, 기후 예측

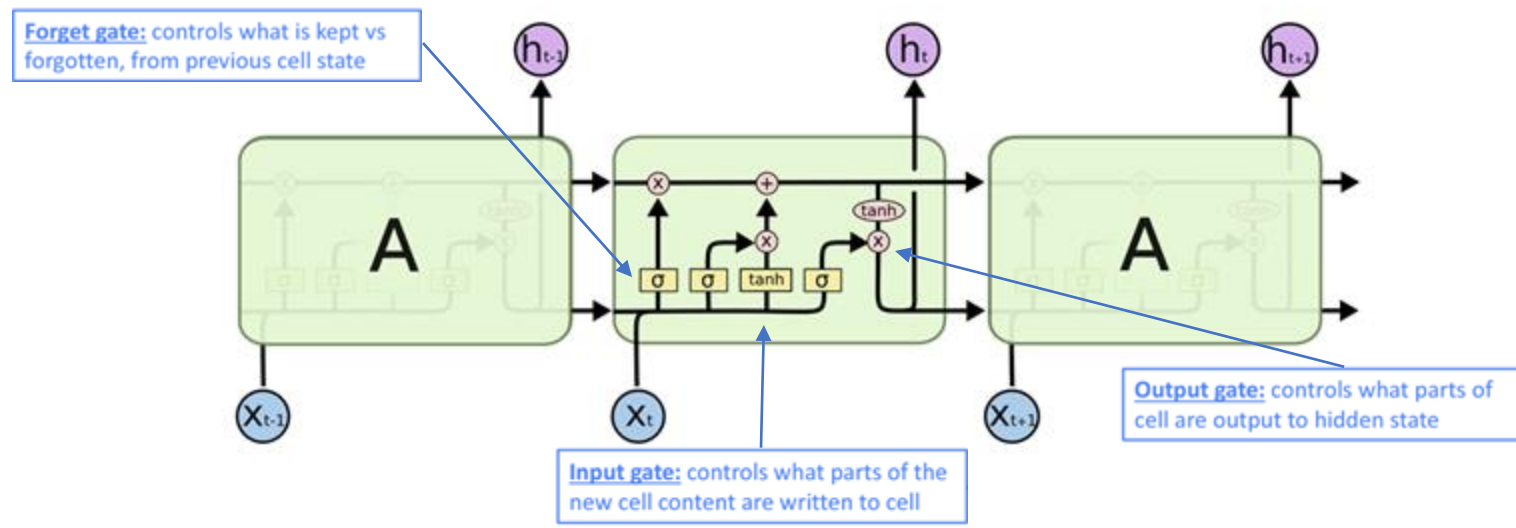
LSTM

- LSTM
 - Long Short-Term Memory
 - RNN의 한계(장기 의존성 문제)를 극복하기 위해 고안, RNN의 일종



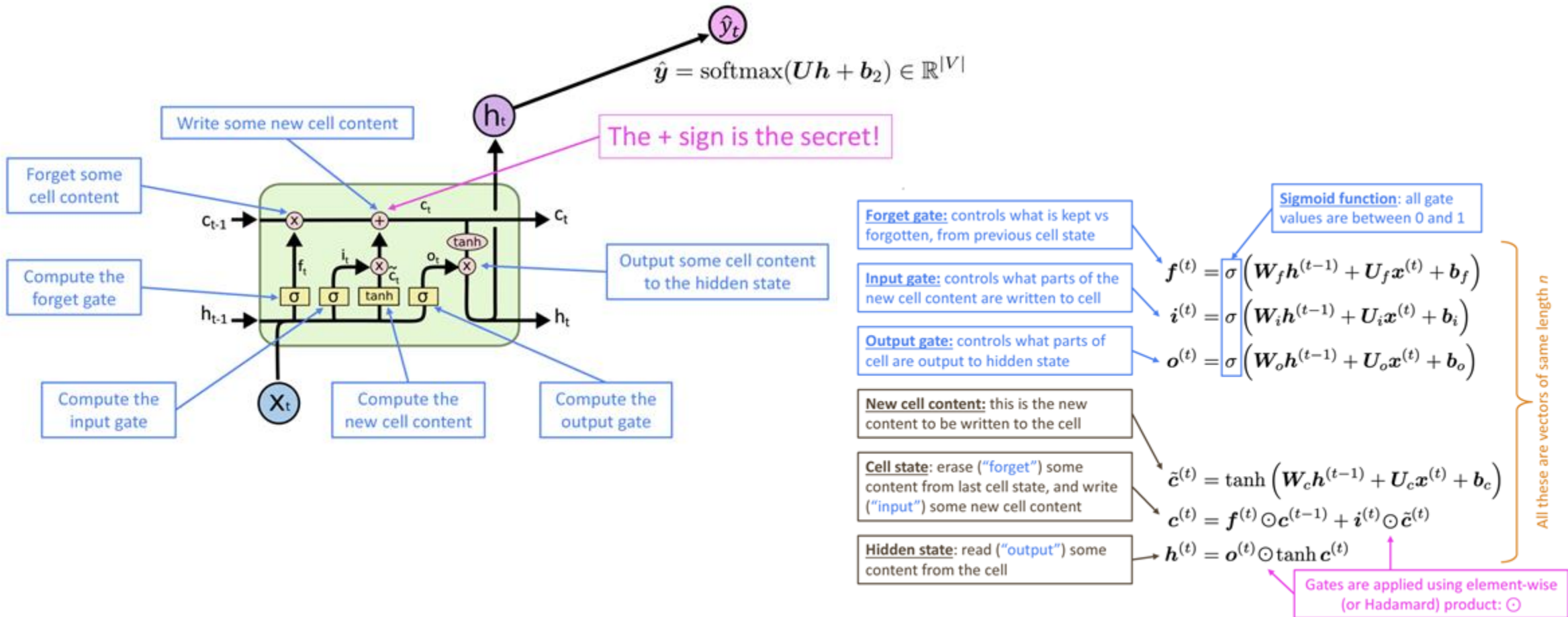
LSTM

- **Hidden State**: 출력하는 값이자, 다음 시점으로 전달되는 **단기 메모리**
- **Cell state**: 장기적으로 기억해야 할 정보를 유지하는 **장기 메모리**
- Gates: 정보를 선택적으로 통과시키는 gate
 - **input gate** - 새로운 정보를 얼마나 반영할 것인지 결정
 - **forget gate** - 이전 cell state의 정보를 얼마나 잊을 것인지 결정
 - **output gate** - 현재 cell state를 기반으로 출력할 정보를 결정



LSTM

- LSTM의 동작

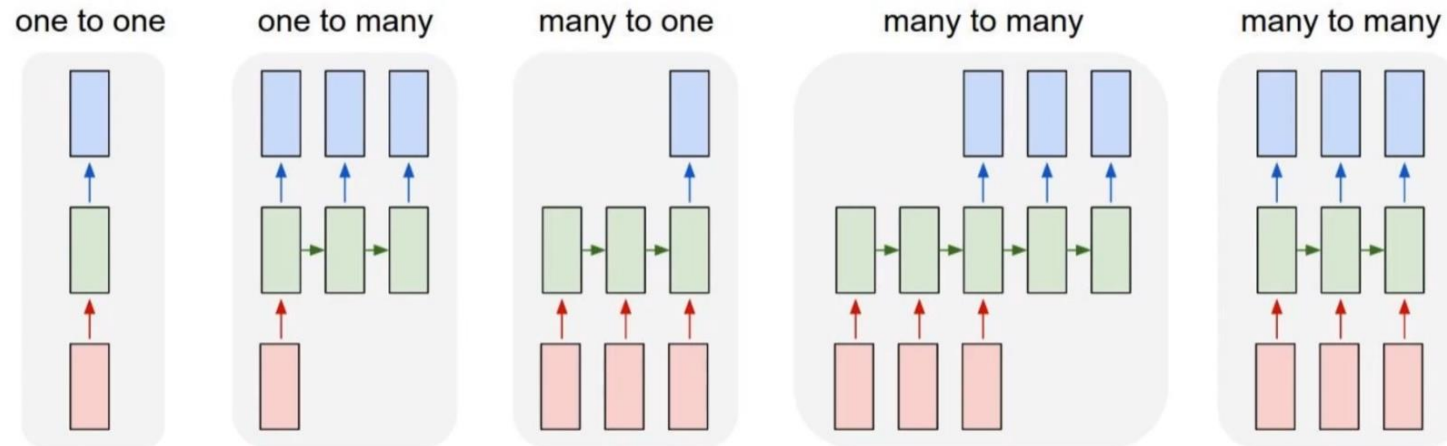


LSTM

- LSTM의 장점
 - 장기 의존성 문제 해결
 - **cell state**와 **gate mechanism**을 통해 중요한 정보를 장기적으로 유지함
 - 다양한 응용 가능
 - 텍스트 생성, 번역, 음성 인식, 시계열 예측과 같은 작업들에 유용함

Task 2.1

- RNN 구현 및 실행 해보기
- 목표
 - 문장을 하나 학습시킨 후, 문장의 일부를 주었을 때 나머지 문장을 잘 예측하는지 확인해보기
 - One-to-one 구조 코드를 보고, Many-to-one 구조 구현해보기
 - 둘 간의 차이점 확인해보기



Task 2.2

- LSTM 실행해보기
- 목표
 - IMDb 데이터셋으로 모델을 학습시킨 후, 임의의 리뷰를 주었을 때 그 리뷰가 긍정인지 부정인지 판단해보기
 - 참고 : IMDb dataset - 영화 사이트 리뷰 데이터 (긍정-부정 판단)

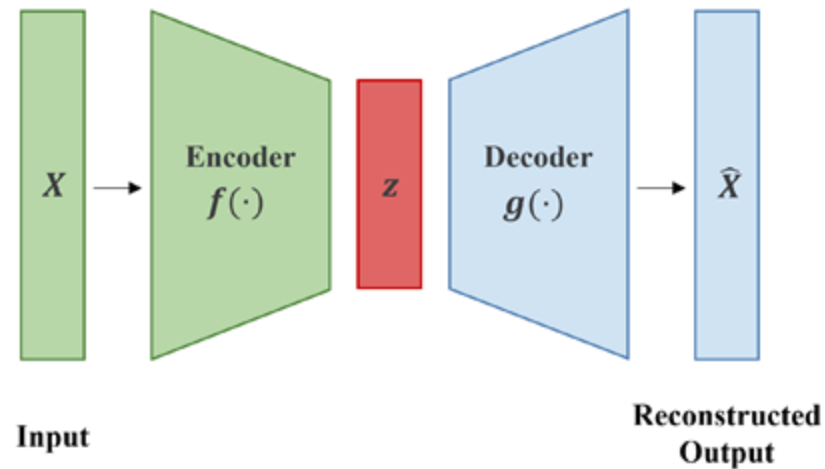
Neural Machine Translation

- Neural network을 통해서 번역 task를 수행하는 것
- 기본적인 RNN – LSTM 구조는 **입력 길이와 출력 길이가 동일해야 모델을 쉽게 설계할 수 있음**
→ 번역 task에 적합하지 않다.

We need a new architecture!

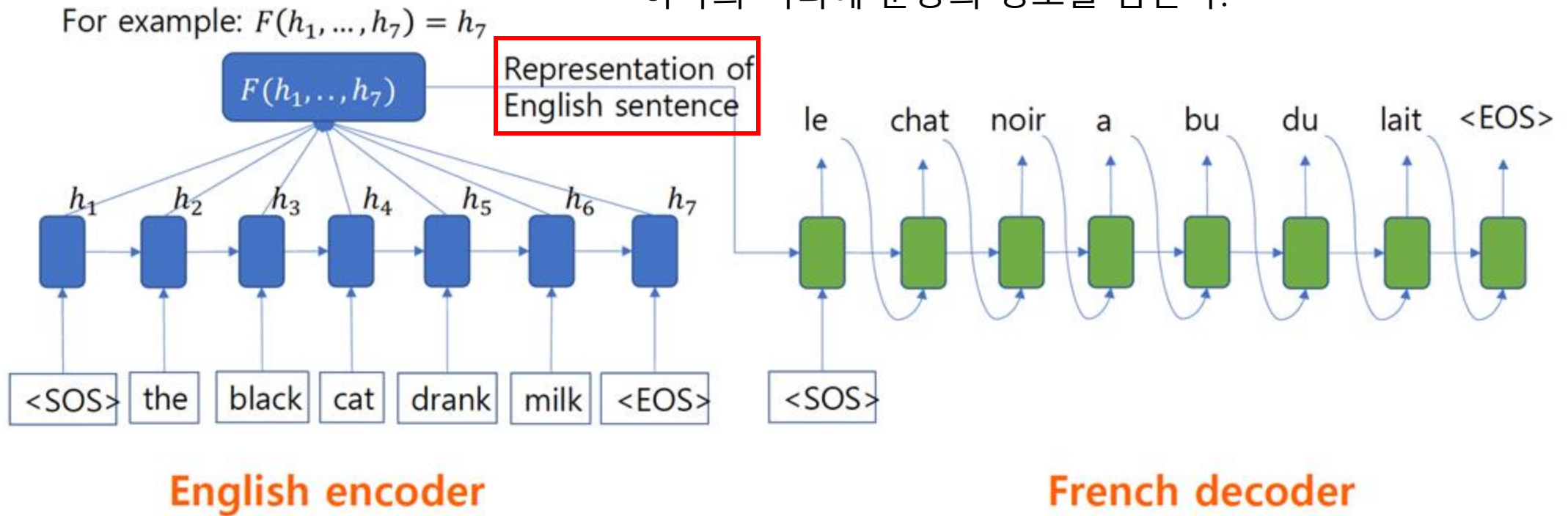
Moving onto Seq2Seq

- Encoder – Decoder 아키텍처
- 기존과 달리 input sequence 전체의 정보(맥락)를 통해 **output sequence**를 생성

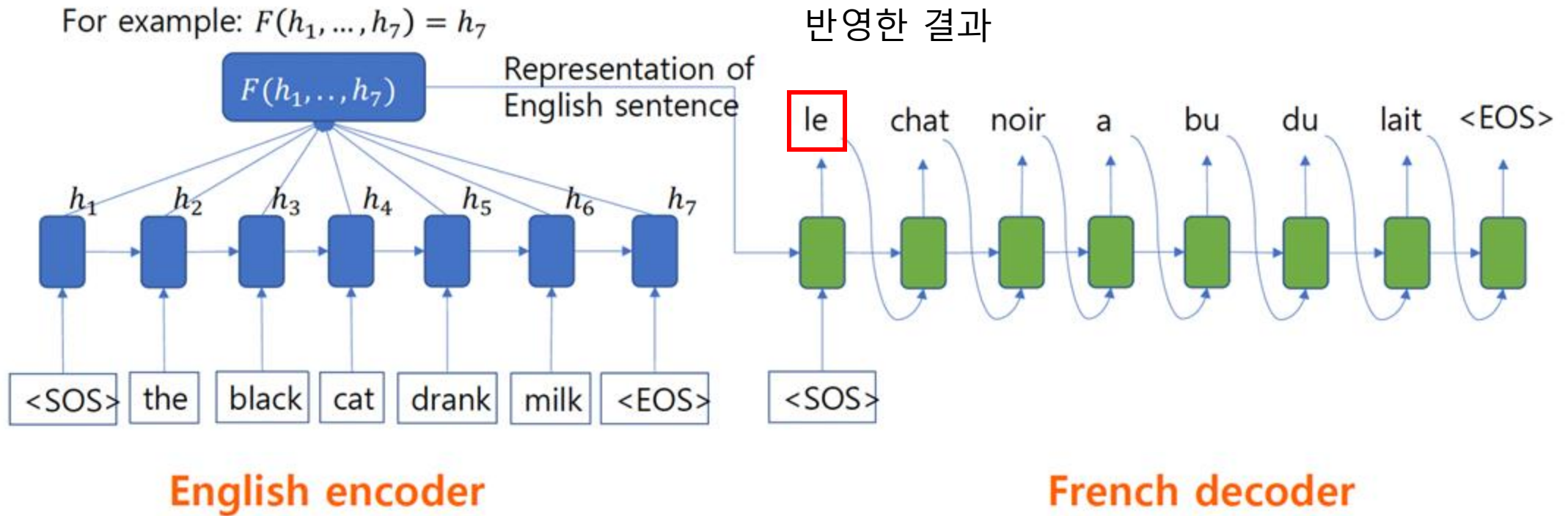


Seq2Seq

하나의 벡터에 문장의 정보를 담는다!



Seq2Seq



How can we evaluate?

- BLEU (BiLingual Evaluation Understudy) score
 - **n-gram** overlap between machine translation output and reference translation
 - **n = 1 ~ 4**
 - mostly used at **evaluating machine translation**

$$\text{BLEU} = \min \left(1, \frac{\text{output-length}}{\text{reference-length}} \right) \left(\prod_{i=1}^4 \text{precision}_i \right)^{\frac{1}{4}}$$

How can we evaluate?

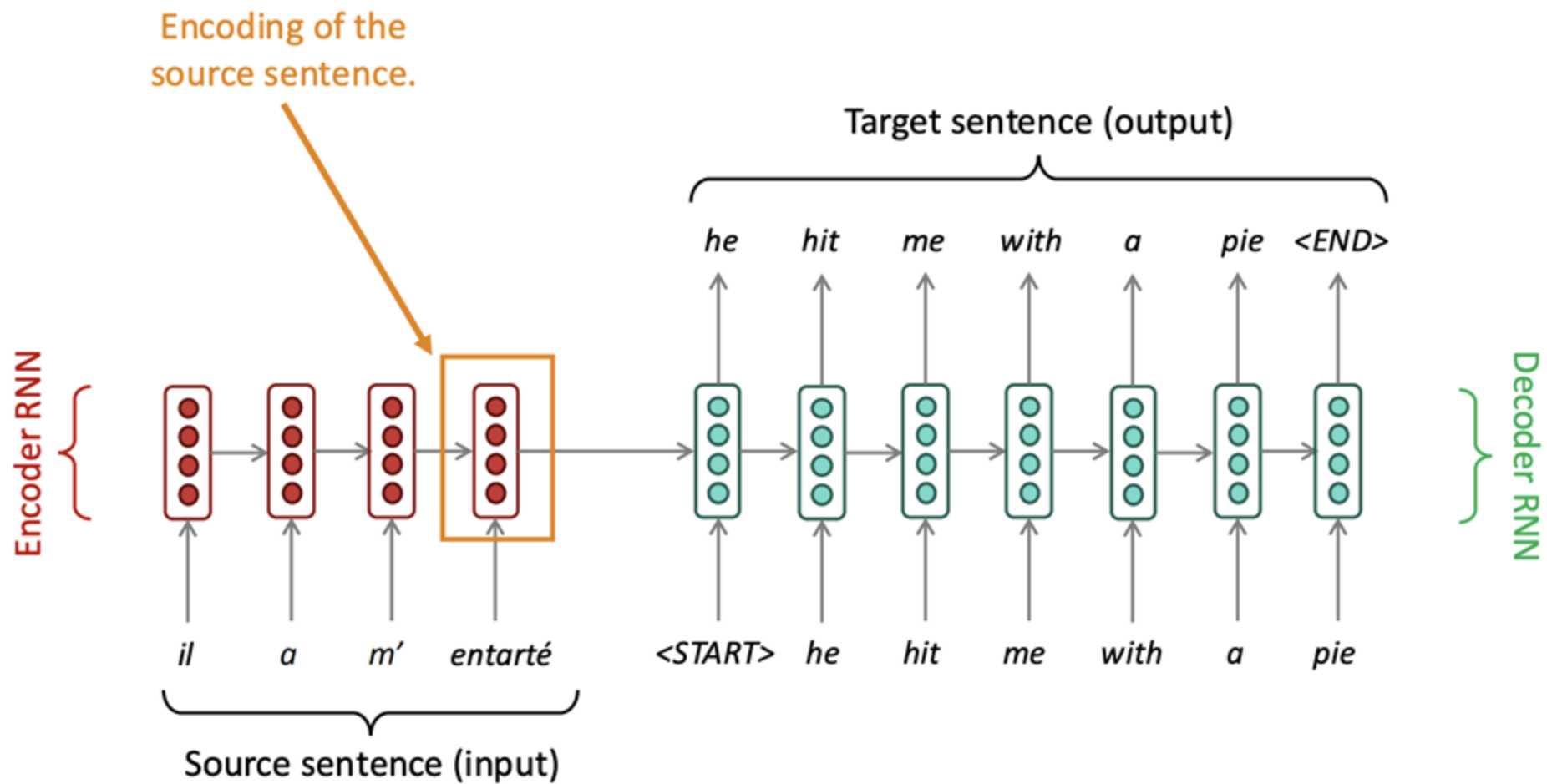
- Sentences
 - Reference (정답): "the cat is on the mat"
 - Candidate (생성): "the cat the cat on the mat"
- N-gram
 - 1-gram: "the" "cat" "on" "mat"
 - 2-gram: "the cat" "on the" "the mat"
 - 3-gram: "on the mat"
 - 4-gram: x

Benchmark

- What is Benchmark?
 - 모델의 성능을 평가하기 위해 표준화된 테스트나 데이터셋을 사용하는 것
 - GLUE, SuperGLUE, SQUAD 등
 - Metric : BLEU score, Accuracy

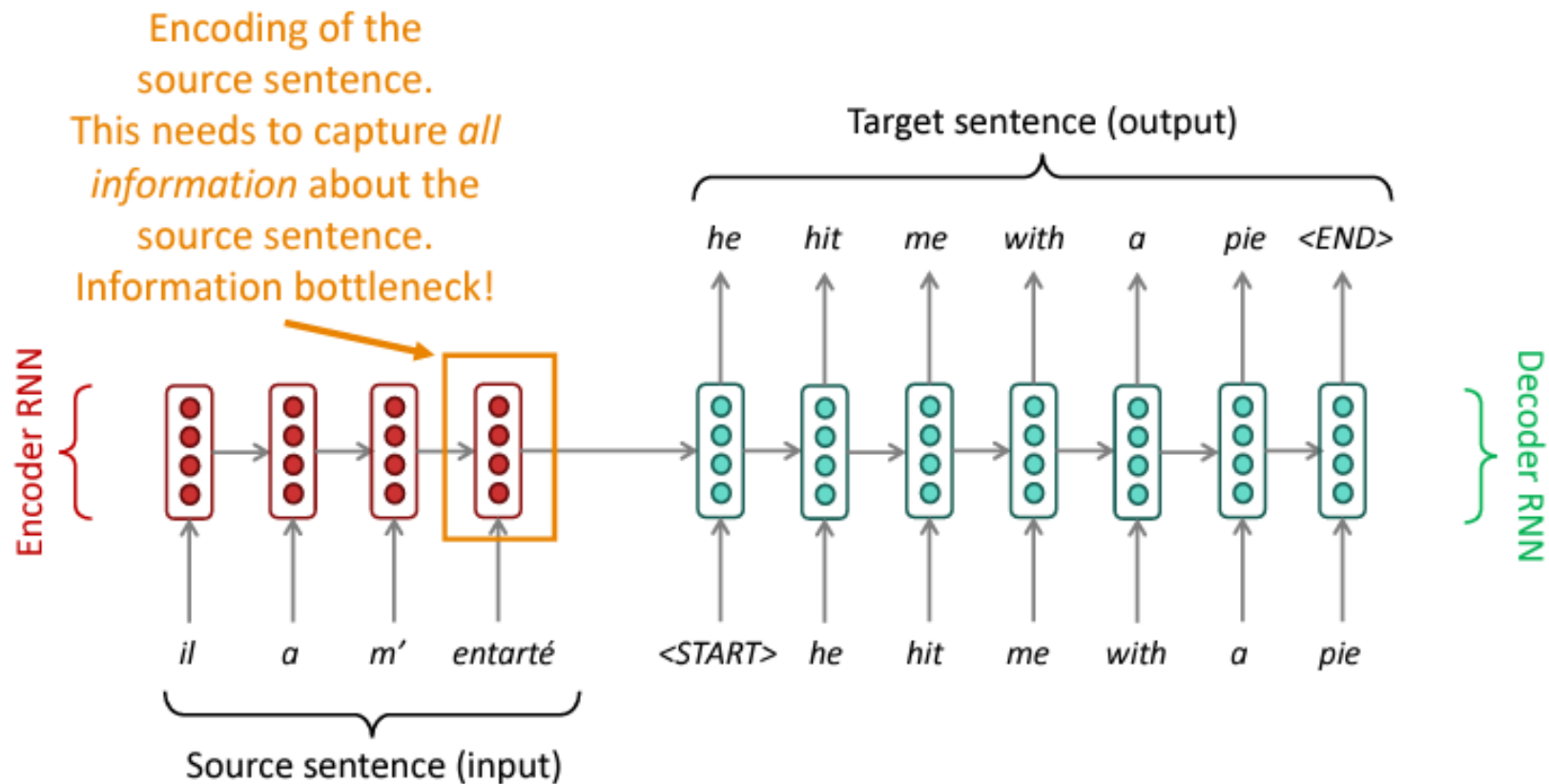


한계



한계

- The bottleneck problem



New Paradigm “Attention”

- Attention provides a solution to the bottleneck problem.
 - **Core idea** : Decoder의 각 단계에서 Encoder와 직접 연결하여 input 시퀀스의 특정 부분에 가중치를 줘보자
- 문장의 중요한 부분에 집중하자!



Attention

- 번역(출력) 시에 문장의 특정 부분에 집중(Attention)하는 방법 사용

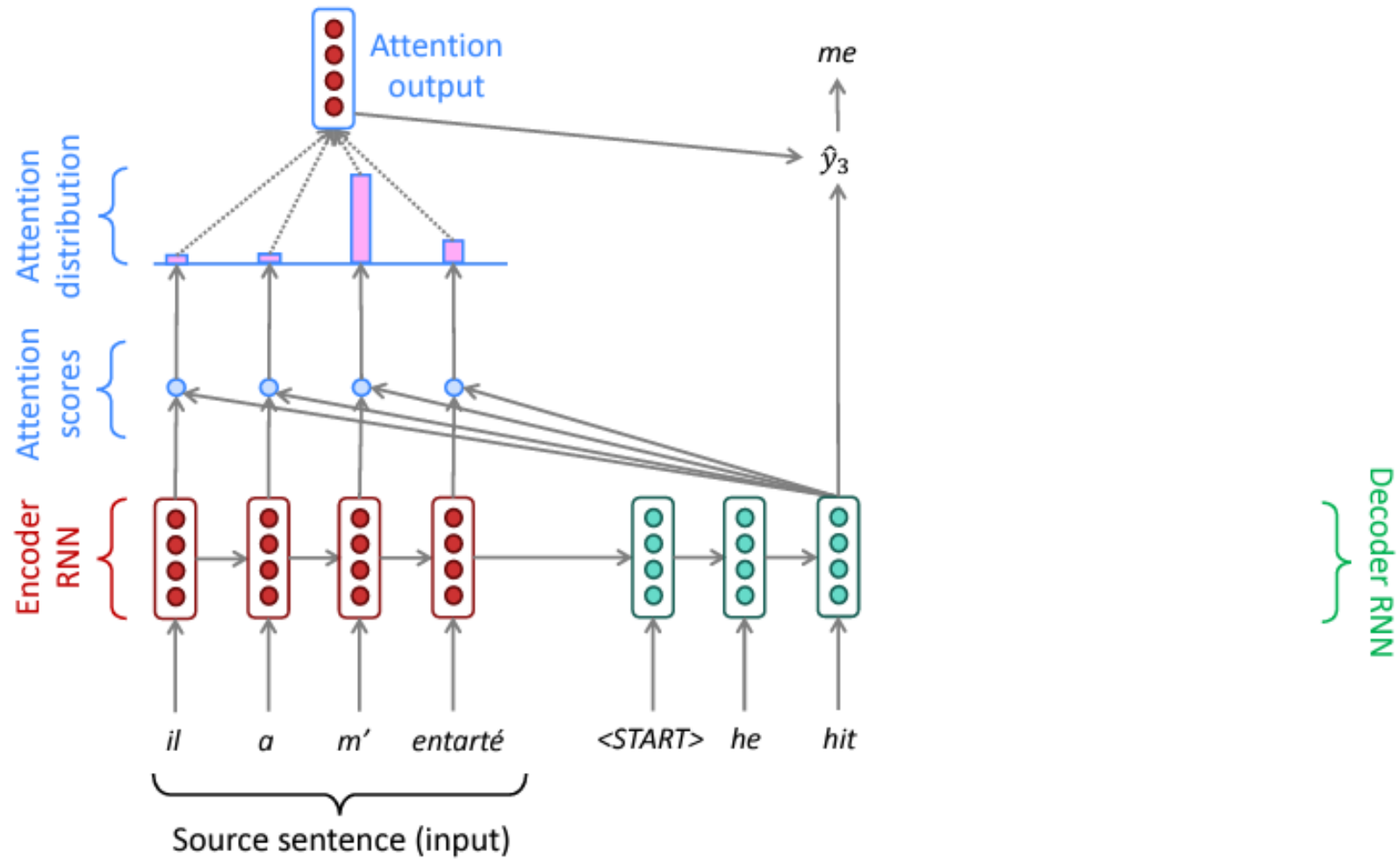
번역(Machine translation) 예시

“어제 카페 갔었어 거기 사람 많더라”

- ‘**cafe**’라는 단어를 생성하기 위해 유사도가 높은 ‘**카페**’, ‘**거기**’ 등의 단어에 Attention 하여 단어를 생성한다.

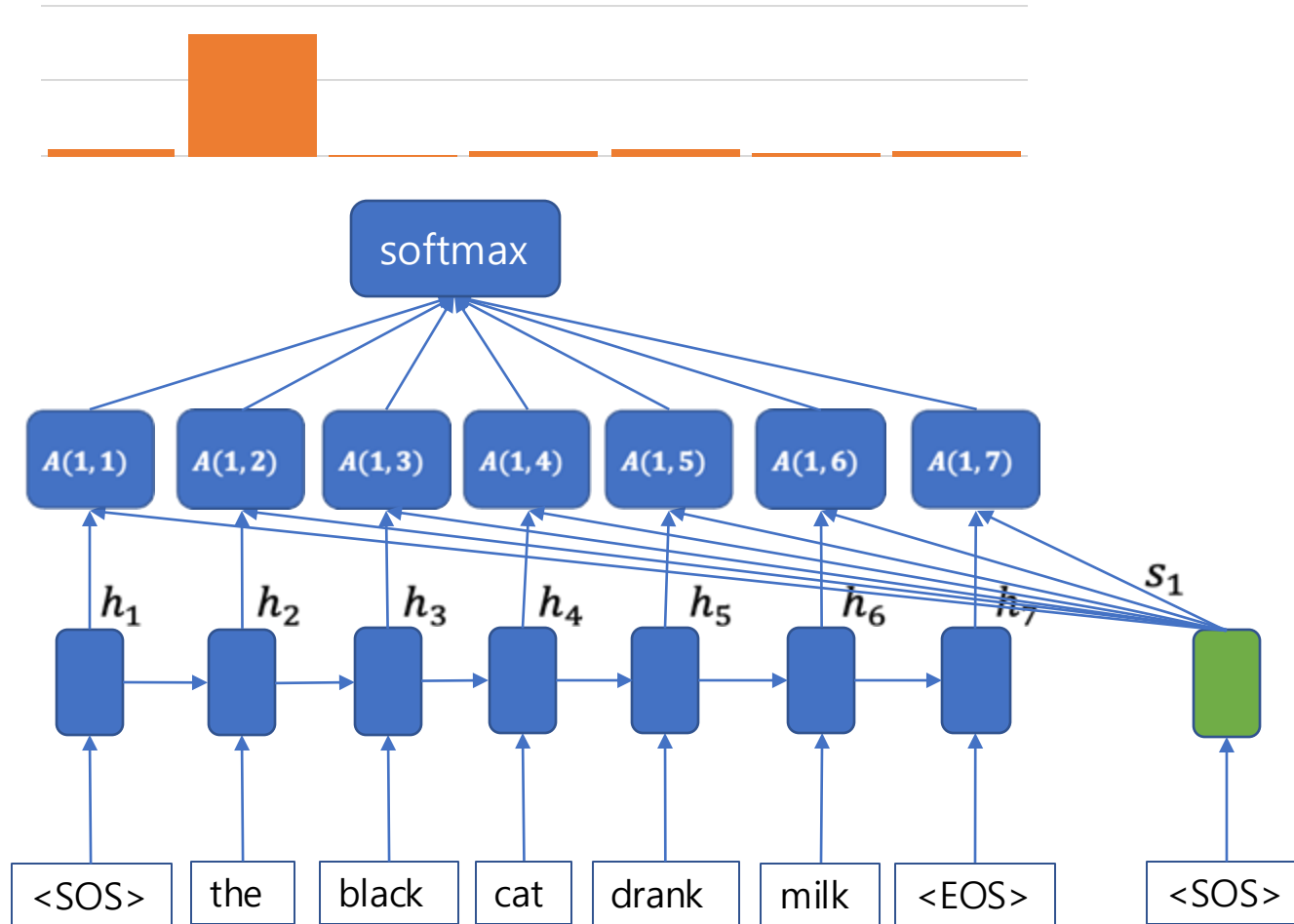


Attention



Attention

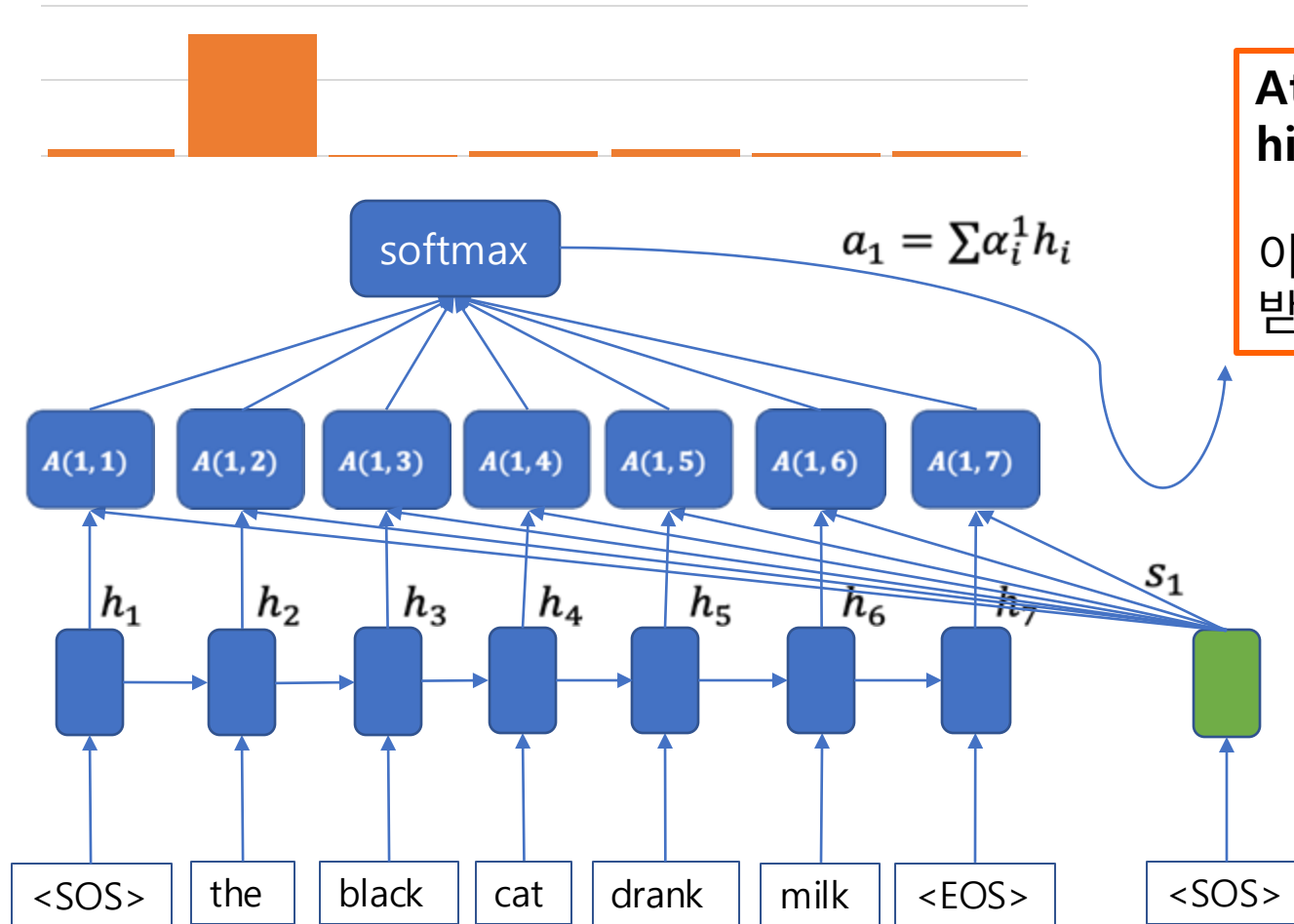
On this decoder timestep, we're mostly focusing on the second encoder hidden state ("the")



- h_i : i 번째 **encoder hidden state**
- s_t : t 번째 **decoder hidden state** (timestep t)
- $score(s_t, h_i) = e_i^t$: decoder hidden state와 encoder hidden state의 유사도 (**attention 점수**)
- $\alpha_i^t = softmax(e_i^t)$: 첫 번째 decoder step에서의 **attention distribution** (**softmax score**), i 번째 encoder 위치에 대한 집중 정도
- a_t : 첫 decoder timestep의 **context vector**

Attention

On this decoder timestep, we're mostly focusing on the second encoder hidden state ("the")



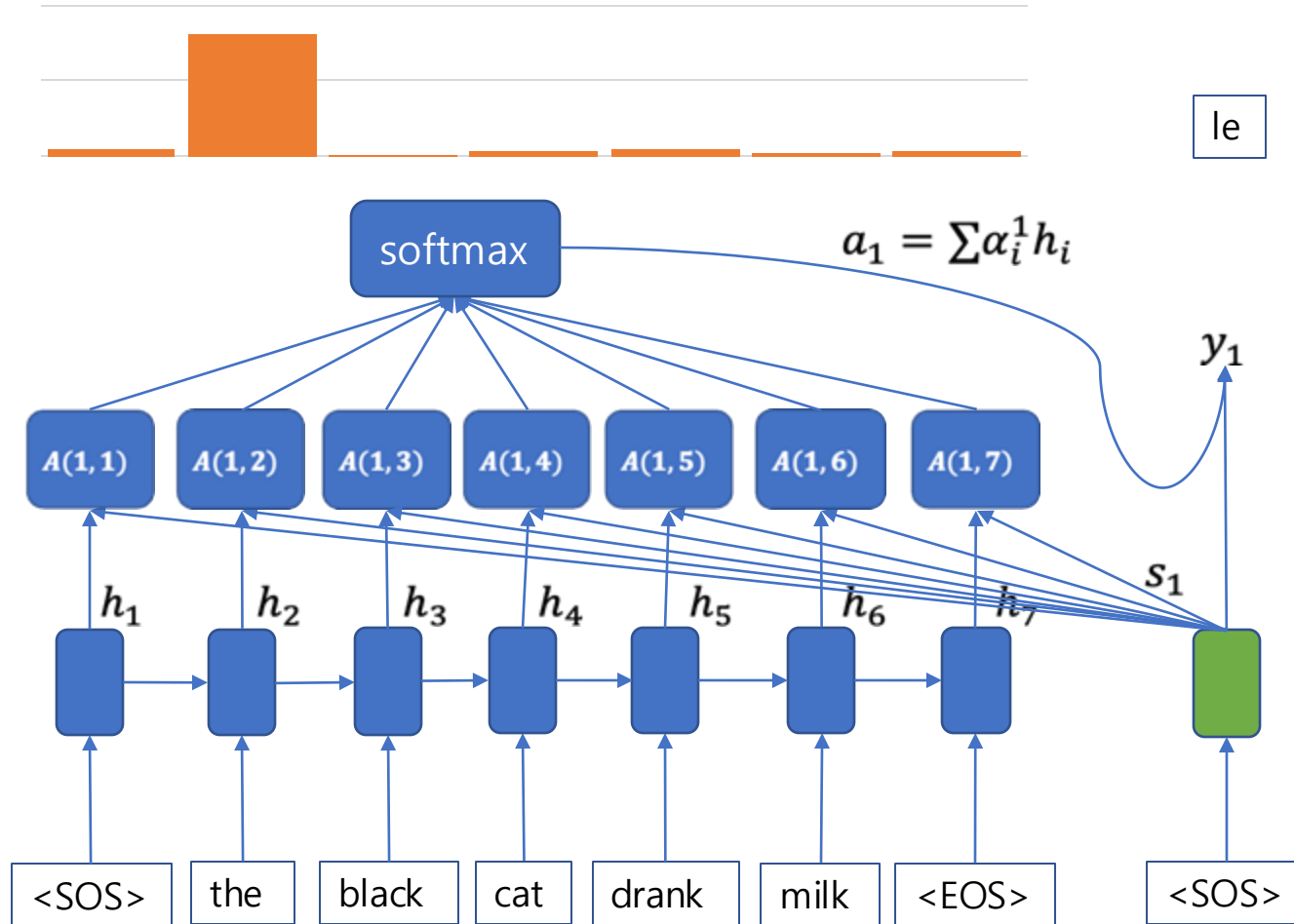
Attention distribution을 사용해 encoder hidden states의 weighted sum을 계산

이때 생성된 attention output은 high attention을 받은 hidden states의 정보를 주로 반영

- h_i : i 번째 encoder hidden state
- s_t : t 번째 decoder hidden state (timestep t)
- $\text{score}(s_t, h_i) = e_i^t$: decoder hidden state와 encoder hidden state의 유사도 (attention 점수)
- $\alpha_i^t = \text{softmax}(e_i^t)$: 첫 번째 decoder step에서의 attention distribution (softmax score), i 번째 encoder 위치에 대한 집중 정도
- a_i : 첫 decoder timestep의 context vector

Attention

On this decoder timestep, we're mostly focusing on the second encoder hidden state ("the")



We hope to produce this output:

le chat noir a bu du lait <EOS>

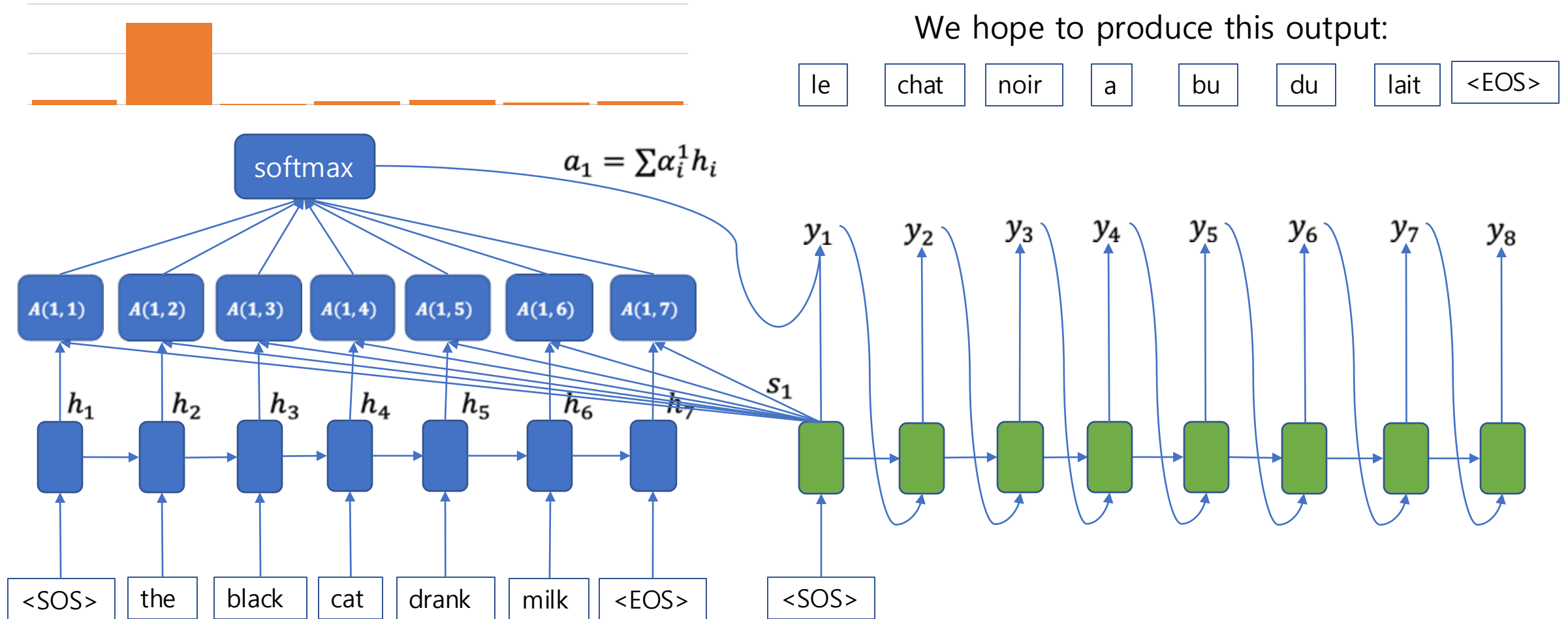
Attention output과 decoder hidden state를 concatenate한 뒤, 이를 이용해 y_1 계산

$$\tilde{s}_1 = \tanh(W_c[a_1; s_1])$$

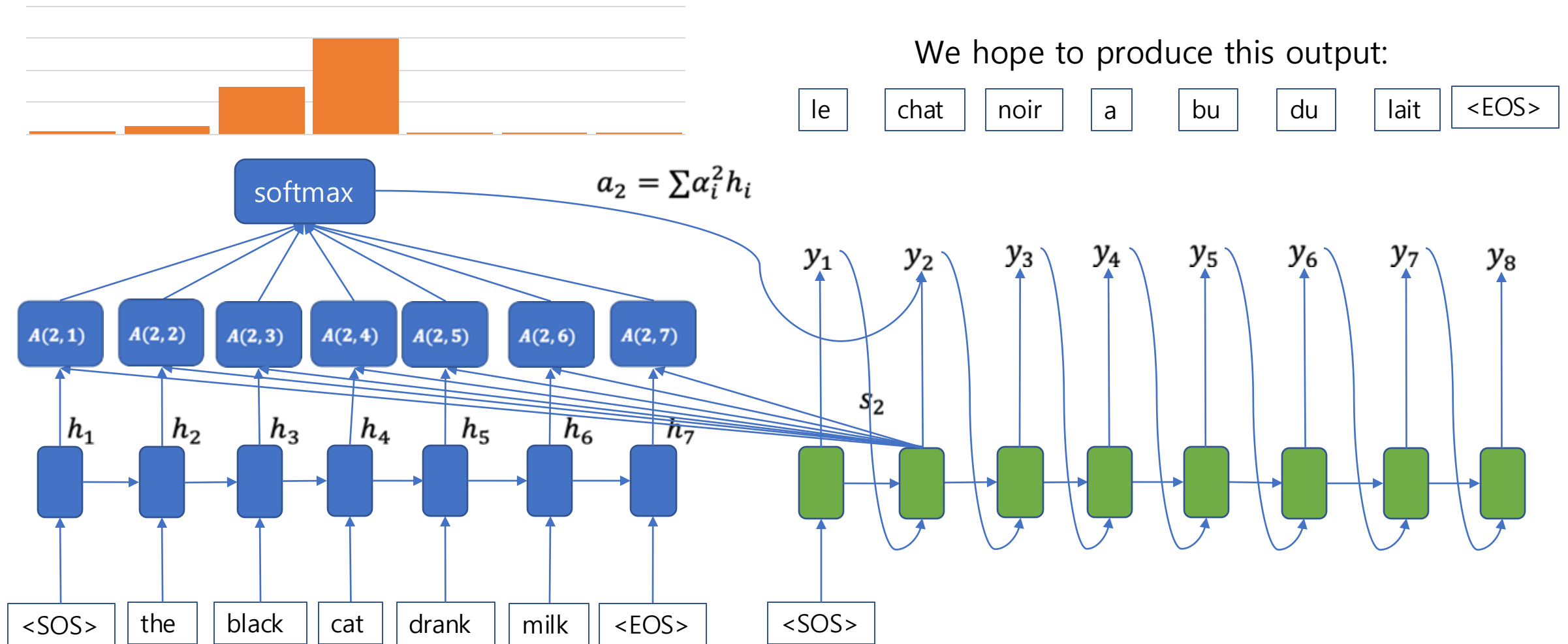
$$y_1 = \text{softmax}(W_s \tilde{s}_1 + b_s)$$

Attention

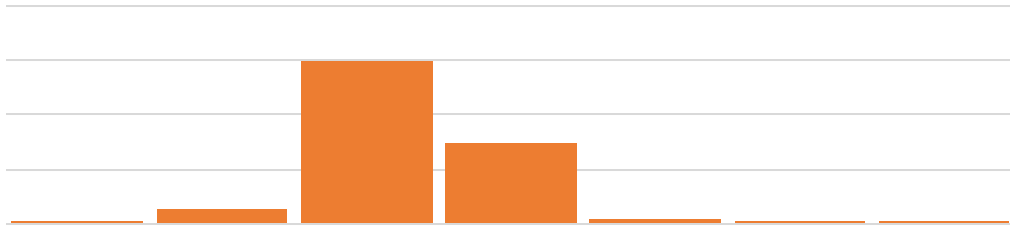
On this decoder timestep, we're mostly focusing on the second encoder hidden state ("the")



Attention

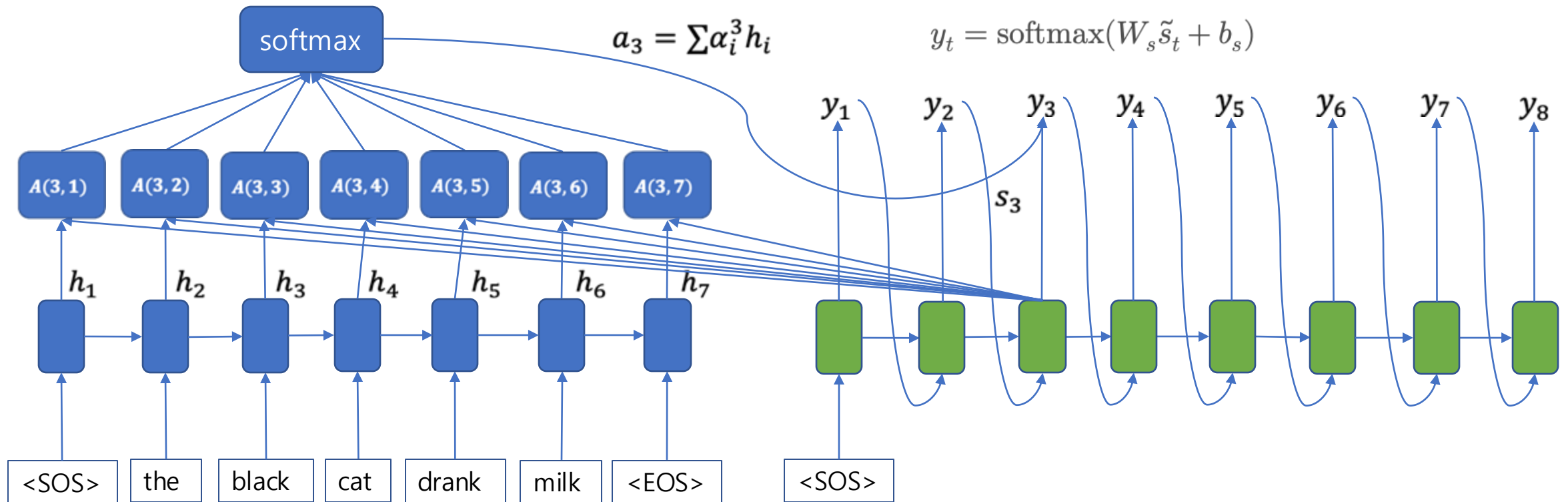


Attention



We hope to produce this output:

le chat noir a bu du lait <EOS>



Task 3

- 목표
 - LSTM & RNN으로 seq2seq 구조로 영어-프랑스어 번역기를 만들어 보기

Practice 5

- 목표
 - Seq2Seq으로 영어-한국어 번역기 만들어 보기

Task 4

- Attention을 사용해 영어-한국어 번역기를 만들어보기

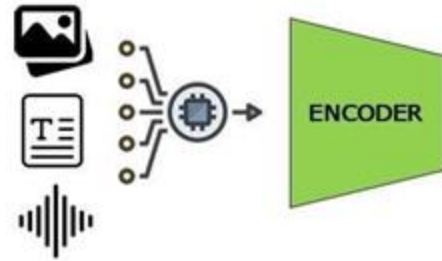
Task 5

- We are now in multimodal era!
- Multimodal : 텍스트, 이미지, 음성 등 다양한 형태의 데이터를 통합하여 이해하고 처리하는 방식
 - 입력받은 Image를 이해해서 Text를 출력할 수는 없을까?

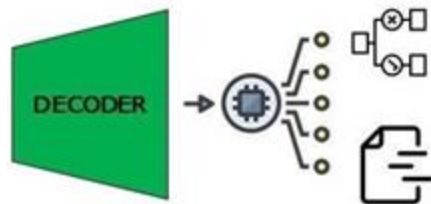
Task 5

- Image Captioning with LSTM
 - VGG16으로 이미지 특징을 추출하고, LSTM으로 캡션을 생성하는 이미지 captioning 모델 설계

- VGG16 encoder



- LSTM decoder



Task 5

- Image Captioning with LSTM
 - VGG16으로 이미지 특징을 추출하고, LSTM으로 캡션을 생성하는 이미지 captioning 모델 설계
- Image : Flickr8k_Dataset
- Let's test with image!

Next Lecture: Language Model after Transformer

Thank you